

Analyse MVVM et CRUD - Projet SuiviPortefolio

Date : 02/10/2026

Auteur : Mistral Vibe (Assistant IA)

Projet : SuiviPortefolio (Application WPF de suivi de portefeuille financier)

Technologies : C#, WPF, .NET, SQLite, MVVM, Repository Pattern

Sommaire

1. [Introduction](#)
 2. [Analyse MVVM](#)
 - [Points forts](#)
 - [Problèmes et axes d'amélioration](#)
 3. [Analyse CRUD](#)
 - [Points forts](#)
 - [Problèmes et axes d'amélioration](#)
 4. [Remarques générales et bonnes pratiques](#)
 5. [Recommandations globales](#)
 6. [Exemples d'amélioration](#)
 - [CompteViewModel](#)
 - [CompteRepository](#)
 - [CompteEditWindow](#)
 7. [Résumé des actions prioritaires](#)
 8. [Conclusion](#)
-

Introduction

Ce document présente une **analyse détaillée** du projet **SuiviPortefolio** sous les angles **MVVM (Model-View-ViewModel)** et **CRUD (Create, Read, Update, Delete)**. L'objectif est d'identifier les **points forts**, les **problèmes** et les **axes d'amélioration** pour rendre le code plus **maintenable**, **testable**, **robuste** et **conforme aux bonnes pratiques**.

Le projet est une application **WPF** en **C#** utilisant **SQLite** pour la persistance des données. Il gère des **comptes bancaires**, des **portefeuilles**, des **positions** et des **actifs financiers**. L'architecture suit globalement le pattern **MVVM** avec un **Repository Pattern** pour l'accès aux données.

Analyse MVVM

Le pattern **MVVM** (Model-View-ViewModel) vise à **découpler** la logique métier de l'interface utilisateur. Dans une application WPF, cela permet de :

- **Séparer** la logique de l'UI.
- **Faciliter les tests unitaires** (pas de dépendance à l'UI).
- **Améliorer la maintenabilité** (chaque couche a une responsabilité claire).

Points forts

1. Séparation claire des responsabilités

Ton architecture respecte globalement la séparation MVVM :

Couche	Dossier/Classe	Responsabilité
Model	Comptes/CompteModel/Compte.cs	Classes métiers (POCO) sans logique.
Model	Comptes/CompteModel/CompteExposition.cs	Modèle de données pour les expositions.
View	Comptes/CompteView/CompteView.xaml.cs	Gestion de l'UI et des événements.
View	Comptes/CompteView/CompteEditWindow.xaml.cs	Fenêtre d'édition des comptes.
ViewModel	Comptes/CompteViewModel/CompteViewModel.cs	Logique métier, état, commandes.
Repository	Comptes/CompteRepository/CompteRepository.cs	Accès aux données (abstraction de SQLite).

 **Bon point** : Le **ViewModel ne connaît pas la View** (découplage via `INotifyPropertyChanged` et `ICommand`).

2. Implémentation correcte de `INotifyPropertyChanged`

- **CompteViewModel** implémente `INotifyPropertyChanged` et notifie les changements de propriétés (`CompteSelectionne`, `ValorisationStatus`).
- Les propriétés utilisent le **pattern standard** avec *backing fields* et `OnPropertyChanged()`.

Exemple :

```

public Compte? CompteSelectionne
{
    get => _compteSelectionne;
    set
    {
        if (_compteSelectionne != value)
        {
            _compteSelectionne = value;
            OnPropertyChanged();
            ((RelayCommand)ModifierCompteCommand).RaiseCanExecuteChanged();
            ((RelayCommand)SetCompteDefaultCommand).RaiseCanExecuteChanged();
        }
    }
}

```

✅ **Bon point** : La gestion des notifications est **propre et fonctionnelle**.

⚠️ **Amélioration possible** : Utiliser [CallerMemberName] pour éviter de répéter le nom de la propriété.

3. Utilisation de RelayCommand pour les commandes

- Tu as implémenté RelayCommand pour gérer les ICommand.
- Les commandes sont bien typées (Func<object?, Task>, Predicate<object?> pour CanExecute).

Exemple :

```

AjouterCompteCommand = new RelayCommand(async _ => await AjouterCompteAsync());
ModifierCompteCommand = new RelayCommand(async _ => await ModifierCompteAsync(), _ =>
CompteSelectionne != null);

```

✅ **Bon point** : Les commandes sont **asynchrones** et gèrent CanExecute.

❌ **Problème : Duplication de code** (voir [Problème 3](#)).

4. Injection de dépendances (partiellement)

- Le constructeur de CompteViewModel accepte un ICompteRepository? (optionnel).
- Si aucun n'est fourni, il crée une instance concrète de CompteRepository.

Exemple :

```
public CompteViewModel(ICompteRepository? repository = null)
{
    _repository = repository ?? new CompteRepository(DatabaseLocation.DatabasePath);
    // ...
}
```

✅ **Bon point** : C'est une **bonne approche pour faciliter les tests unitaires**.

⚠️ **Amélioration possible** : Éviter le `new` dans le ViewModel (utiliser un conteneur DI).

❌ Problèmes et axes d'amélioration

1. DataContext créé directement dans la View

Fichier : `CompteView.xaml.cs`

Code :

```
public CompteView()
{
    InitializeComponent();
    DataContext = new CompteViewModel(); // ❌ Problème
}
```

⚠️ **Problèmes** :

- **Couplage fort** entre View et ViewModel (la View connaît la classe concrète).
- **Difficile à tester** : Impossible de mock le ViewModel pour des tests UI.
- **Pas d'injection de dépendances** : Le ViewModel est créé sans contrôle.

✅ **Solution** : Utiliser un **ViewModel Locator** ou **injecter le ViewModel via le conteneur DI**.

Exemple avec DI (`App.xaml.cs`) :

```

public partial class App : Application
{
    public IServiceProvider ServiceProvider { get; private set; }

    protected override void OnStartup(StartupEventArgs e)
    {
        var services = new ServiceCollection();
        services.AddSingleton<ICompteRepository, CompteRepository>();
        services.AddSingleton<CompteViewModel>();
        ServiceProvider = services.BuildServiceProvider();

        var mainWindow = new MainWindow { DataContext =
        ServiceProvider.GetRequiredService<CompteViewModel>() };
        mainWindow.Show();
    }
}

```

2. CompteEditWindow gère directement la logique métier

Fichier : CompteEditWindow.xaml.cs

Code :

```

private void Enregistrer_Click(object sender, RoutedEventArgs e)
{
    if (string.IsNullOrEmpty(NomTextBox.Text)) { ... }
    Compte.CpteNom = NomTextBox.Text.Trim(); // ❌ Logique dans la View
    DialogResult = true;
}

```

⚠ Problèmes :

- **La View contient de la logique métier** (validation, modification de l'objet Compte).
- **Pas de binding vers un ViewModel dédié** : Les champs (NomTextBox, SoldeTextBox) ne sont pas liés à des propriétés d'un ViewModel.

✅ **Solution** : Créer un **ViewModel pour CompteEditWindow** qui gère :

- Les propriétés (Nom, Solde, Devise, etc.) avec `INotifyPropertyChanged`.
- La validation (via `INotifyDataErrorInfo` ou des propriétés `IsValid`).
- La commande `EnregistrerCommand`.

Exemple :

```
public class CompteEditViewModel : INotifyPropertyChanged
{
    private string _nom;
    public string Nom
    {
        get => _nom;
        set { _nom = value; OnPropertyChanged(); Validate(); }
    }
    // ...
}
```

3. Duplication de RelayCommand

⚠ Problème :

- RelayCommand est défini **2 fois** :
 1. Dans Comptes/CompteViewModel/CompteViewModel.cs (version locale).
 2. Dans Consultations/ConsultationsViewModel/RelayCommand.cs.

✅ **Solution** : Déplacer RelayCommand dans un dossier **Infrastructure** ou **Common** et le réutiliser partout.

Exemple de structure :

```
SuiviPortefolio/
├── Infrastructure/
│   ├── RelayCommand.cs
│   └── ObservableObject.cs
└── ...
```

4. Pas de découplage complet entre les couches

⚠ Problèmes :

- **CompteViewModel dépend directement de CompteRepository** (même si ICompteRepository existe).
- **CompteViewModel dépend de CotationFetcher** (instancié en dur : `private readonly CotationFetcher _cotationFetcher = new();`).

✅ **Solution** :

- **Injecter ICompteRepository et ICotationFetcher** via le constructeur.
- **Créer une interface ICotationFetcher** pour découpler l'appel à Yahoo Finance.

Exemple :

```
public interface ICotationFetcher
{
    Task<Cotation?> FetchSnapshotAsync(string symbol);
}

public class CompteViewModel : INotifyPropertyChanged
{
    private readonly ICompteRepository _repository;
    private readonly ICotationFetcher _cotationFetcher;

    public CompteViewModel(ICompteRepository repository, ICotationFetcher cotationFetcher)
    {
        _repository = repository;
        _cotationFetcher = cotationFetcher;
    }
}
```

5. Pas de gestion centralisée des erreurs dans le ViewModel

⚠ Problèmes :

- Les exceptions dans `ChargerValorisationsAsync` sont capturées mais pas toujours propagées proprement.
- Pas de mécanisme pour afficher les erreurs à l'utilisateur (autre que `ValorisationStatus`).

✅ Solution :

- Utiliser un **service de logging** (ex: `ILogger`).
- Implémenter un **message bus** (ex: `IMessenger` de `CommunityToolkit.Mvvm`) pour notifier les erreurs à la View.

6. ObservableCollection est public et mutable

Code :

```
public ObservableCollection<Compte> Comptes { get; } = new();
```

⚠ Problème :

- La collection est **modifiable de l'extérieur** (même si `get;` est en lecture seule, la référence peut être gardée).
- **Pas de contrôle sur les modifications** (ex: validation avant ajout).

✓ **Solution :**

- Rendre la collection **private set;** et exposer des méthodes pour l'ajout/suppression.
 - Ou utiliser **ReadOnlyObservableCollection** pour exposer une vue en lecture seule.
-

7. Pas de tests unitaires

⚠ **Problème :**

- Aucun fichier de test n'est visible dans la structure (dossier Tests existe mais semble vide).
- Sans tests, difficile de valider les modifications et d'assurer la robustesse.

✓ **Solution :** Ajouter des tests pour :

- `CompteViewModel` (logique CRUD, validation).
- `CompteRepository` (requêtes SQL).
- `RelayCommand` (exécution, `CanExecute`).

Outils recommandés :

- **xUnit** ou **NUnit** (frameworks de test).
 - **Moq** (pour les mocks).
-

8. Utilisation de `async void` pour `Execute` dans `RelayCommand`

Code :

```
public async void Execute(object? parameter) => await _execute(parameter);
```

⚠ **Problème :**

- `async void` est **dangereux** car les exceptions ne peuvent pas être capturées par un `try/catch` classique.
- **Les exceptions sont propagées au `SynchronizationContext`** et peuvent crash l'application.

✓ **Solution :** Utiliser `async Task` et gérer les exceptions dans `Execute` :

```
public void Execute(object? parameter)
{
    try
    {
        _executeAsync(parameter).ConfigureAwait(false).GetAwaiter().GetResult();
    }
    catch (Exception ex)
    {
        // Log l'erreur
        Debug.WriteLine(ex);
        throw; // ou affiche une erreur à l'utilisateur
    }
}
```

Analyse CRUD

Le **CRUD** (Create, Read, Update, Delete) est un ensemble d'opérations de base pour manipuler des données. Ton code implémente ces opérations via le **Repository Pattern**, mais il y a des points à améliorer.

Points forts

1. Séparation des responsabilités

- **Repository** (CompteRepository) gère **uniquement** l'accès aux données (SQLite).
- **ViewModel** (CompteViewModel) gère la logique métier et appelle le Repository.

 **Bon point** : Le Repository est **agnostique** de l'UI et du ViewModel.

2. Implémentation complète des opérations

Opération	Méthode	Statut
Create	Add(Compte compte)	✅ Implémentée
Read	GetAll(), GetExpositions()	✅ Implémentées
Update	Update(Compte compte)	✅ Implémentée
Delete	Delete(int id)	✅ Implémentée
Custom	SetDefault(int id)	✅ Implémentée

✅ **Bon point** : Toutes les opérations CRUD sont présentes.

3. Utilisation de transactions pour SetDefault

Code :

```
using var transaction = connection.BeginTransaction();
// ...
transaction.Commit();
```

✅ **Bon point** : Gestion correcte des transactions pour les opérations atomiques.

4. Requêtes SQL bien structurées

- Utilisation de **requêtes paramétrées** (\$nom, \$type, etc.) pour éviter les injections SQL.
- **LEFT JOIN** dans GetExpositions() pour récupérer les valorisations des actifs.

✅ **Bon point** : Code SQL **propre et sécurisé**.

⚠ **Amélioration possible** : Optimiser les requêtes complexes (ex: GetExpositions).

❌ Problèmes et axes d'amélioration

1. GetAll() charge toutes les données en mémoire

Code :

```
public List<Compte> GetAll()
{
    var result = new List<Compte>();
    // ...
    while (reader.Read()) { result.Add(new Compte { ... }); }
    return result;
}
```

⚠ Problèmes :

- **Pas de pagination** : Si la table `Compte` devient très grande, cela peut poser des problèmes de performance.
- **Pas de filtres** : Impossible de filtrer les comptes sans tout charger.

✅ **Solution** : Ajouter des méthodes comme :

```
List<Compte> GetByPtId(int ptId);
List<Compte> GetByType(TypeCompte type);
Compte? GetById(int id);
```

Implémenter la **pagination** si nécessaire.

2. Pas de gestion des erreurs dans le Repository

⚠ Problèmes :

- Les exceptions SQL (ex: `SqliteException`) ne sont **pas capturées** dans `CompteRepository`.
- Si une requête échoue, l'exception remonte directement au `ViewModel`.

✅ **Solution** : Capturer les exceptions dans le Repository et :

- **Logger l'erreur** (ex: `logger.LogError(ex, "Erreur lors de la récupération des comptes")`).
- **Retourner une liste vide ou null** (avec un mécanisme pour notifier l'erreur).
- **Ou utiliser un `Result<T>`** (pattern pour gérer les erreurs de manière fonctionnelle).

Exemple :

```

public List<Compte> GetAll()
{
    try
    {
        // Logique actuelle
    }
    catch (SqliteException ex)
    {
        _logger?.LogError(ex, "Erreur SQL dans GetAll");
        throw new DataAccessException("Impossible de récupérer les comptes", ex);
    }
}

```

3. Delete(int id) ne vérifie pas l'existence du compte

Code :

```

public void Delete(int id)
{
    using var connection = new SqliteConnection(_connectionString);
    connection.Open();
    using var command = connection.CreateCommand();
    command.CommandText = "DELETE FROM Compte WHERE CpteId = $id;";
    command.Parameters.AddWithValue("$id", id);
    command.ExecuteNonQuery(); // ❌ Pas de vérification
}

```

⚠️ **Problèmes :**

- **Pas de feedback** : On ne sait pas si le compte a été supprimé.
- **Risque de suppression de données inexistantes** (pas grave, mais pas propre).

✅ **Solution** : Retourner un **bool** (true si supprimé, false sinon) ou utiliser un **Result<bool>**.

Exemple :

```
public bool Delete(int id)
{
    using var connection = new SqliteConnection(_connectionString);
    connection.Open();
    using var command = connection.CreateCommand();
    command.CommandText = "DELETE FROM Compte WHERE CpteId = $id;";
    command.Parameters.AddWithValue("$id", id);
    return command.ExecuteNonQuery() > 0;
}
```

4. Pas de gestion des contraintes d'intégrité

⚠ Problèmes :

- `Delete(int id)` ne vérifie pas si le compte est référencé ailleurs (ex: dans `Position`).
- `SetDefault(int id)` ne vérifie pas si le compte existe.

✅ **Solution** : Ajouter des vérifications avant suppression :

```

public bool Delete(int id)
{
    using var connection = new SqlConnection(_connectionString);
    connection.Open();

    // Vérifie si le compte existe
    using (var checkCmd = connection.CreateCommand())
    {
        checkCmd.CommandText = "SELECT COUNT(*) FROM Compte WHERE CpteId = $id;";
        checkCmd.Parameters.AddWithValue("$id", id);
        if ((long)checkCmd.ExecuteScalar() == 0)
            return false;
    }

    // Vérifie si le compte est référencé
    using (var checkCmd = connection.CreateCommand())
    {
        checkCmd.CommandText = "SELECT COUNT(*) FROM Position WHERE PosCpteId = $id;";
        checkCmd.Parameters.AddWithValue("$id", id);
        if ((long)checkCmd.ExecuteScalar() > 0)
            throw new InvalidOperationException("Le compte ne peut pas être supprimé car il contient des positions.");
    }

    // Suppression
    using (var deleteCmd = connection.CreateCommand())
    {
        deleteCmd.CommandText = "DELETE FROM Compte WHERE CpteId = $id;";
        deleteCmd.Parameters.AddWithValue("$id", id);
        return deleteCmd.ExecuteNonQuery() > 0;
    }
}

```

5. Add(Compte compte) ne gère pas les conflits

⚠ Problème :

- Si un CpteNom dupliqué est inséré, SQLite va lever une exception (si UNIQUE est défini).
- Pas de **validation côté Repository**.

✅ **Solution** : Ajouter une validation dans le Repository :

```

public int Add(Compte compte)
{
    if (string.IsNullOrEmpty(compte.CpteNom))
        throw new ArgumentException("Le nom du compte est obligatoire.");

    // Vérifie si le nom existe déjà
    using var connection = new SqlConnection(_connectionString);
    connection.Open();
    using var checkCmd = connection.CreateCommand();
    checkCmd.CommandText = "SELECT COUNT(*) FROM Compte WHERE CpteNom = $nom;";
    checkCmd.Parameters.AddWithValue("$nom", compte.CpteNom);
    if ((long)checkCmd.ExecuteScalar() > 0)
        throw new InvalidOperationException("Un compte avec ce nom existe déjà.");

    // Insertion
    // ...
}

```

6. Pas de méthode GetById

⚠ Problème :

- **CompteViewModel** utilise GetAll() puis filtre avec FirstOrDefault :

```

CompteSelectionne = Comptes.FirstOrDefault(compte => compte.CpteId ==
nouveauCompteId);

```

✅ **Solution** : Ajouter une méthode GetById(int id) dans ICompteRepository :

```

public Compte? GetById(int id)
{
    using var connection = new SqlConnection(_connectionString);
    connection.Open();
    using var command = connection.CreateCommand();
    command.CommandText = "SELECT * FROM Compte WHERE CpteId = $id;";
    command.Parameters.AddWithValue("$id", id);
    using var reader = command.ExecuteReader();
    return reader.Read() ? new Compte { /* ... */ } : null;
}

```

7. GetExpositions() est complexe et peu flexible

⚠ Problèmes :

- La requête SQL dans GetExpositions() est **longue et difficile à maintenir**.
- Elle utilise un LEFT JOIN avec GROUP BY qui peut être **inefficace**.

✅ Solution :

- **Découper la requête** en plusieurs méthodes si nécessaire.
 - **Utiliser des vues SQL** (si la requête est souvent utilisée).
 - **Optimiser avec des indexes** sur PosCpteId et PosActifId.
-

8. Pas de gestion des null dans les modèles

⚠ Problème :

- Dans CompteExposition, ActifDevise est nullable (string?), mais pas les autres propriétés.
- Dans CompteValorisation, ValorisationActions et SoldeTotal sont nullable.
- **Risque de NullReferenceException** si une propriété nullable est accédée sans vérification.

✅ Solution :

- **Utiliser le pattern ?.** pour accéder aux propriétés nullable.
 - **Ajouter des validations** pour éviter les null.
-

9. Pas de migration de base de données

⚠ Problème :

- La base SQLite est initialisée via InitializeDatabase() dans CompteRepository.
- **Pas de système de migrations** (ex: Entity Framework Core).

✅ Solution :

- Utiliser **Entity Framework Core** (même en SQLite) pour gérer les migrations.
 - Ou implémenter un **système de migrations manuel** (ex: fichiers SQL versionnés).
-

📌 Remarques générales et bonnes pratiques

✅ Points forts généraux

1. **Code bien structuré** : Les dossiers Model, ViewModel, View, Repository sont clairs.

2. **Nommage cohérent** : Les propriétés (CpteNom, CpteSolde) et méthodes (GetAll, Add) sont bien nommées.
 3. **Utilisation de `async/await`** : Les méthodes asynchrones (ChargerValorisationsAsync) sont bien implémentées.
 4. **Gestion des tâches longues** : `_isRefreshingValorisations` évite les exécutions concurrentes.
-

✖ Problèmes généraux

1. Pas de `IDisposable` pour `HttpClient`

⚠ Problème :

- `HttpClient` (dans `CotationFetcher`) n'est pas géré correctement.
- Actuellement, il est probablement instancié à chaque appel (risque de **socket exhaustion**).

✅ Solution : Rendre `HttpClient` singleton :

```
private static readonly HttpClient _httpClient = new();
public CotationFetcher() : this(_httpClient) { }
public CotationFetcher(HttpClient httpClient) { _httpClient = httpClient; }
```

2. Pas de validation côté modèle

⚠ Problèmes :

- `Compte` n'a **aucune validation** (ex: `CpteNom` peut être vide, `CpteSolde` peut être négatif).
- `CompteEditWindow` gère la validation **dans le code-behind** (✖).

✅ Solution :

- **Ajouter des validations dans le modèle** (ex: avec `INotifyDataErrorInfo` ou des méthodes `Validate`).
- **Utiliser `FluentValidation`** (library populaire pour la validation en .NET).

Exemple avec `FluentValidation` :

```
public class CompteValidator : AbstractValidator<Compte>
{
    public CompteValidator()
    {
        RuleFor(x => x.CpteNom).NotEmpty().MaxLength(100);
        RuleFor(x => x.CpteDevise).NotEmpty().Length(3);
        RuleFor(x => x.CpteSolde).GreaterThanOrEqualTo(0);
    }
}
```

3. Pas de logging

⚠ Problème :

- **Aucun système de logging** n'est visible dans le code.
- Les erreurs sont **capturées mais pas loggées** (ex: dans ChargerValorisationsAsync).

✅ **Solution** : Utiliser `Microsoft.Extensions.Logging` ou **Serilog**.

Exemple :

```
private readonly ILogger<CompteViewModel> _logger;

public CompteViewModel(ICompteRepository repository, ILogger<CompteViewModel> logger)
{
    _logger = logger;
    // ...
}

try
{
    // Code
}
catch (Exception ex)
{
    _logger.LogError(ex, "Erreur lors du chargement des valorisations");
    ValorisationStatus = "Erreur : " + ex.Message;
}
```

4. Pas de gestion des cultures (CultureInfo)

⚠ Problème :

- Dans `CompteEditWindow`, tu utilises `CultureInfo.CurrentCulture` pour formater le solde :

```
SoldeTextBox.Text = compte.CpteSolde.ToString("0.00", CultureInfo.CurrentCulture);
```

- **Incohérence entre affichage et saisie** (ex: affiche 123,45 mais attend 123.45 en entrée).

✅ Solution :

- **Utiliser une culture fixe** (ex: `CultureInfo.InvariantCulture`) pour les données numériques.
 - **Ou gérer la conversion** entre la culture de l'UI et la culture des données.
-

5. Pas de gestion des annulations (Cancel dans les fenêtres modales)

⚠ Problème :

- Dans `CompteEditWindow`, si l'utilisateur clique sur **Annuler**, la fenêtre se ferme mais **rien n'est fait**.

✅ Solution :

- **Annuler les modifications** si l'utilisateur clique sur **Annuler**.
 - **Fermer proprement** la fenêtre.
-

6. Pas de Equals/GetHashCode dans les modèles

⚠ Problème :

- `Compte` et `CompteExposition` n'implémentent pas `Equals` et `GetHashCode`.
- **Comportement imprévisible** si ces objets sont utilisés dans des `HashSet` ou `Dictionary`.

✅ Solution : Implémenter `Equals` et `GetHashCode` (ou utiliser des **records** en C# 9+).

Exemple :

```
public class Compte : IEquatable<Compte>
{
    public int CpteId { get; set; }
    // ...

    public override bool Equals(object? obj) ⇒ Equals(obj as Compte);

    public bool Equals(Compte? other) ⇒
        other ≠ null &&
        CpteId == other.CpteId;

    public override int GetHashCode() ⇒ CpteId.GetHashCode();
}
```

7. Pas de documentation XML

⚠ Problème :

- **Aucun commentaire XML** pour les classes/méthodes publiques.
- **Difficile de comprendre l'API** sans lire le code.

✅ **Solution** : Ajouter des **commentaires XML** pour les classes/méthodes publiques.

Exemple :

```
/// <summary>
/// Représente un compte bancaire ou un portefeuille.
/// </summary>
public class Compte
{
    /// <summary>
    /// Identifiant unique du compte.
    /// </summary>
    public int CpteId { get; set; }
    // ...
}
```

Recommandations globales

Catégorie	Recommandation	Priorité	Difficulté
MVVM	Centraliser RelayCommand dans un fichier unique	★★★★★	★
MVVM	Utiliser un ViewModel Locator ou DI	★★★★★	★★
MVVM	Créer un ViewModel pour CompteEditWindow	★★★★	★★
MVVM	Injecter ICompteRepository et ICotationFetcher	★★★★	★★
CRUD	Ajouter GetById, Exists, IsReferenced dans ICompteRepository	★★★★	★
CRUD	Gérer les erreurs dans le Repository	★★★★	★★
CRUD	Ajouter des validations dans CompteRepository	★★★	★★
Bonnes pratiques	Utiliser HttpClient comme singleton	★★★★★	★
Bonnes pratiques	Ajouter des tests unitaires	★★★★	★★★
Bonnes pratiques	Ajouter un système de logging	★★★★	★★
Bonnes pratiques	Utiliser CommunityToolkit.Mvvm	★★★	★★

Exemples d'amélioration

CompteViewModel

1. Interface ICotationFetcher

```
public interface ICotationFetcher
{
    Task<Cotation?> FetchSnapshotAsync(string symbol);
}
```

2. Injection des dépendances

```

public class CompteViewModel : INotifyPropertyChanged
{
    private readonly ICompteRepository _repository;
    private readonly ICotationFetcher _cotationFetcher;
    private readonly ILogger<CompteViewModel> _logger;

    public CompteViewModel(
        ICompteRepository repository,
        ICotationFetcher cotationFetcher,
        ILogger<CompteViewModel> logger)
    {
        _repository = repository;
        _cotationFetcher = cotationFetcher;
        _logger = logger;

        AjouterCompteCommand = new RelayCommand(async _ => await AjouterCompteAsync());
        ModifierCompteCommand = new RelayCommand(async _ => await ModifierCompteAsync(), _
=> CompteSelectionne != null);
    }
}

```

3. Utilisation de `CommunityToolkit.Mvvm` (recommandé)

Si tu utilises le package `CommunityToolkit.Mvvm`, tu peux simplifier ton code :

```

public class CompteViewModel : ObservableObject
{
    private readonly ICompteRepository _repository;
    private readonly ICotationFetcher _cotationFetcher;

    [ObservableProperty]
    private Compte? _compteSelectionne;

    [ObservableProperty]
    private string _valorisationStatus = "...";

    [RelayCommand]
    private async Task AjouterCompteAsync()
    {
        var nouveauCompte = new Compte { /* ... */ };
        var dialog = new CompteEditWindow(nouveauCompte, nouveau: true);
        if (dialog.ShowDialog() != true) return;

        var id = _repository.Add(nouveauCompte);
        if (nouveauCompte.CpteEstDefaut) _repository.SetDefault(id);
        ChargerComptes();
    }
}

```

Avantages :

- Plus besoin d'écrire OnPropertyChanged manuellement.
- Plus besoin d'écrire RelayCommand manuellement.
- Code plus lisible et maintenable.

CompteRepository

1. Ajout de méthodes utiles

```
public interface ICompteRepository
{
    // Existantes
    List<Compte> GetAll();
    List<CompteExposition> GetExpositions();
    int Add(Compte compte);
    void Update(Compte compte);
    bool Delete(int id);
    void SetDefault(int id);

    // Nouvelles
    Compte? GetById(int id);
    bool Exists(int id);
    bool IsReferenced(int id);
}
```

2. Gestion des erreurs

```

public Compte? GetById(int id)
{
    try
    {
        using var connection = new SqlConnection(_connectionString);
        connection.Open();
        using var command = connection.CreateCommand();
        command.CommandText = "SELECT * FROM Compte WHERE CpteId = $id;";
        command.Parameters.AddWithValue("$id", id);
        using var reader = command.ExecuteReader();
        return reader.Read() ? MapCompte(reader) : null;
    }
    catch (SqliteException ex)
    {
        _logger.LogError(ex, "Erreur lors de la récupération du compte {Id}", id);
        throw new DataAccessException("Impossible de récupérer le compte", ex);
    }
}

private Compte MapCompte(SqliteDataReader reader)
{
    return new Compte
    {
        CpteId = reader.GetInt32(0),
        CpteNom = reader.GetString(1),
        CpteType = Enum.Parse<TypeCompte>(reader.GetString(2)),
        CpteDevise = reader.GetString(3),
        CpteSolde = reader.GetDecimal(4),
        CpteEstDefaut = reader.GetBoolean(5),
        CptePtfId = reader.GetInt32(6)
    };
}

```

CompteEditWindow

1. ViewModel dédié

```

public class CompteEditViewModel : ObservableObject, IDataErrorInfo
{
    private readonly Compte _compte;
    private readonly bool _isNew;
    private readonly ICompteRepository _repository;

    public CompteEditViewModel(Compte compte, bool isNew, ICompteRepository repository)
    {
        _compte = compte;
        _isNew = isNew;
        _repository = repository;
        Nom = compte.CpteNom;
        Devise = compte.CpteDevise;
        Solde = compte.CpteSolde;
        Type = compte.CpteType;
        EstDefault = compte.CpteEstDefault;
    }

    [ObservableProperty]
    private string _nom;

    [ObservableProperty]
    private string _devise;

    [ObservableProperty]
    private decimal _solde;

    [ObservableProperty]
    private TypeCompte _type;

    [ObservableProperty]
    private bool _estDefault;

    public string Title ⇒ _isNew ? "Nouveau compte" : "Modifier le compte";
    public Array TypesCompte ⇒ Enum.GetValues<TypeCompte>();

    [RelayCommand]
    private void Enregistrer()
    {
        if (!Validate()) return;

        _compte.CpteNom = Nom;
    }
}

```

```

        _compte.CpteDevise = Devise;
        _compte.CpteSolde = Solde;
        _compte.CpteType = Type;
        _compte.CpteEstDefaut = EstDefaut;

        // Ferme la fenêtre avec DialogResult = true
    }

    [RelayCommand]
    private void Annuler()
    {
        // Ferme la fenêtre avec DialogResult = false
    }

    private bool Validate()
    {
        if (string.IsNullOrEmpty(Nom)) return false;
        if (string.IsNullOrEmpty(Devise) || Devise.Length != 3) return false;
        return true;
    }

    public string this[string columnName]
    {
        get
        {
            if (columnName == nameof(Nom) && string.IsNullOrEmpty(Nom))
                return "Le nom est obligatoire";
            if (columnName == nameof(Devise) && (string.IsNullOrEmpty(Devise) ||
Devise.Length != 3))
                return "La devise doit être un code à 3 lettres";
            return string.Empty;
        }
    }

    public string Error => string.Empty;
}

```

2. Code-behind simplifié

```

public partial class CompteEditWindow : Window
{
    public CompteEditWindow(Compte compte, bool nouveau, ICompteRepository repository)
    {
        InitializeComponent();
        DataContext = new CompteEditViewModel(compte, nouveau, repository);
    }
}

```

3. XAML avec Bindings

```

<Window x:Class="SuiviPortefolio.Comptes.CompteView.CompteEditWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="{Binding Title}" Height="300" Width="400">
    <StackPanel Margin="10">
        <TextBox Text="{Binding Nom, UpdateSourceTrigger=PropertyChanged,
ValidatesOnDataErrors=True}" />
        <ComboBox ItemsSource="{Binding TypesCompte}" SelectedItem="{Binding Type}" />
        <TextBox Text="{Binding Devise, UpdateSourceTrigger=PropertyChanged,
ValidatesOnDataErrors=True}" />
        <TextBox Text="{Binding Solde, StringFormat='0.00',
UpdateSourceTrigger=PropertyChanged}" />
        <CheckBox IsChecked="{Binding EstDefault}" Content="Compte par défaut" />
        <StackPanel Orientation="Horizontal" HorizontalAlignment="Right">
            <Button Content="Annuler" Command="{Binding AnnulerCommand}" Margin="0,0,10,0"
Width="80" />
            <Button Content="Enregistrer" Command="{Binding EnregistrerCommand}" Width="80"
/>
        </StackPanel>
    </StackPanel>
</Window>

```

Résumé des actions prioritaires

#	Action	Description	Impact	Difficulté	Effort
1	Centraliser RelayCommand	Éviter la duplication de code	★★★★	★	1h
2	Utiliser un conteneur DI	Découpler les dépendances	★★★★★	★★	2-3h
3	Créer un ViewModel pour CompteEditWindow	Découpler la logique de la View	★★★★	★★	2-4h
4	Injecter ICompteRepository et ICotationFetcher	Améliorer la testabilité	★★★★	★★	1-2h
5	Ajouter GetById, Exists, IsReferenced	Optimiser les requêtes	★★★★	★	1h
6	Gérer les erreurs dans CompteRepository	Robustesse	★★★★	★★	1-2h
7	Utiliser HttpClient comme singleton	Éviter les fuites de ressources	★★★★★	★	30min
8	Ajouter des tests unitaires	Validation	★★★★	★★★	4-8h
9	Ajouter un système de logging	Débogage	★★★	★★	1-2h
10	Utiliser CommunityToolkit.Mvvm	Simplifier le code	★★★	★★	1-2h

Conclusion

Ton code est **déjà très bon** pour un projet personnel. Il respecte **globalement** les principes **MVVM** et **CRUD**, avec une **architecture claire** et une **séparation des responsabilités**.

◆ Points forts à conserver :

- ✓ Séparation claire des couches (Model, ViewModel, View, Repository).
- ✓ Utilisation de INotifyPropertyChanged et ObservableCollection.
- ✓ Implémentation complète des opérations CRUD.
- ✓ Code SQL sécurisé (requêtes paramétrées).

◆ Améliorations prioritaires :

1. **Centraliser RelayCommand** (éviter la duplication).
2. **Utiliser l'injection de dépendances** (pour ICompteRepository, ICotationFetcher, ILogger).
3. **Créer un ViewModel pour CompteEditWindow** (découpler la logique de la View).
4. **Gérer les erreurs dans le Repository** (logging, try/catch).
5. **Ajouter des méthodes utiles dans ICompteRepository** (GetById, Exists, etc.).

◆ Bonus (pour aller plus loin) :

- **Passer à CommunityToolkit.Mvvm** (simplifie INotifyPropertyChanged, RelayCommand, etc.).
- **Ajouter des tests unitaires** (xUnit + Moq).
- **Utiliser Entity Framework Core** (si le projet évolue, pour gérer les migrations).
- **Ajouter un système de logging** (ILogger ou Serilog).



Prochaines étapes

Si tu veux **améliorer ton code**, je te recommande de commencer par :

1. **Centraliser RelayCommand** (action rapide et impact immédiat).
2. **Injecter les dépendances** dans CompteViewModel.
3. **Créer un ViewModel pour CompteEditWindow**.
4. **Ajouter des tests unitaires** pour valider les modifications.

Si tu veux que je t'aide à **implémenter une de ces améliorations**, fais-le-moi savoir ! Je peux te fournir du code prêt à l'emploi ou t'accompagner pas à pas.



Note : Ce document peut être mis à jour au fur et à mesure de l'évolution du projet. N'hésite pas à le compléter avec tes propres remarques ou retours d'expérience !